

A Factor Decomposition Method for Separating Store Substitution and Embedding Placement Effects in Vector Database Migration

벡터 데이터베이스 이전에서 저장소 교체와 임베딩 위치 효과의 분리

2026 International Conference on PyGeek

August 20, 2026 · Hongik University, Seoul

Junha Yoon · Ohhyeon Gwon · Junhaeng Lee · Hyeji Roh · Soowan Cho · Suhee Kim · SangSoon Lim

Sungkyul University · Chung-Ang University · Deep Insight Lab

At a Glance

Quality changed after the migration. Which change caused it?

Problem

- Migrating the vector layer changes the store and the embedding placement at the same time
- A single before-and-after comparison cannot tell which factor is responsible

Proposal

- Insert one intermediate condition — a bridge condition — so that one comparison splits into two
- It needs no re-embedding, so it is inexpensive and leaves the running service untouched

Result

- No quality change was detected for the store substitution; virtually all of the observed change came from the embedding placement

Background · The Vector Layer Changes Most Often

Three components of a RAG system

- Embedding model — turns queries and documents into vectors
- Store — indexes and searches those vectors
- Language model — generates an answer grounded in the retrieved evidence

A representative migration path

- General-purpose relational database with a vector extension → converged database with a vector type and an inference runtime inside it
- That is, the database itself performs the embedding computation

The effect of such a migration is usually assessed by an end-to-end comparison before and after the change.

Problem · Two Things Change at Once

Two factors folded into a single migration

- Store — the engine that holds the vectors and its indexing method (approximate or exact search)
- Placement — whether the embedding is computed outside or inside the database

They change together for a reason

- A migration motivated by governance prefers to move the embedding inside as well
- A database that hosts a model imposes a model size limit, so moving the placement drags the model capacity along with it

The end-to-end difference alone cannot point to the factor that caused the change

Why It Matters · The Cause Gets Misdiagnosed

When quality drops

- It cannot be told apart whether the store or the model is at fault

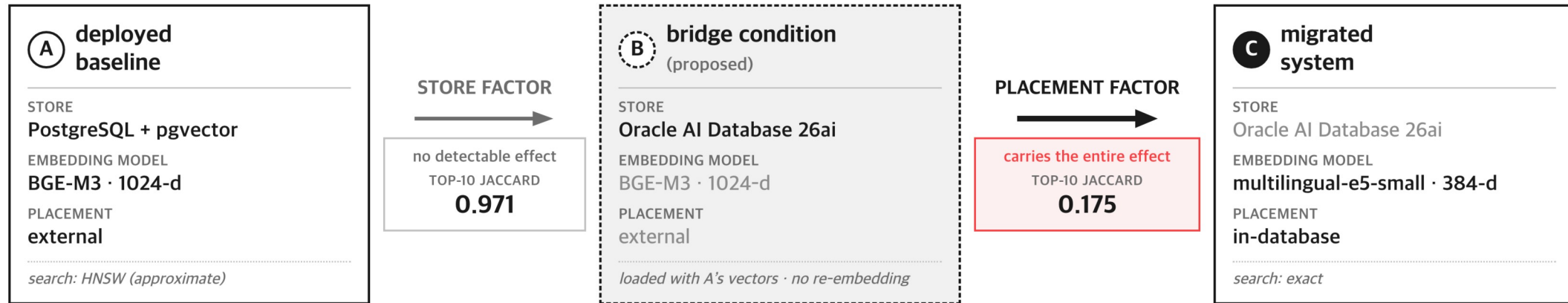
When nothing changes

- It cannot be known whether there was truly no effect, or the two factors cancelled each other out

So the remedy misses

- Blaming the store and reverting it does not bring the quality back
- The conclusion cannot be carried over to an environment with a different instance class or model size limit

Proposal · A Bridge Condition Separates the Factors

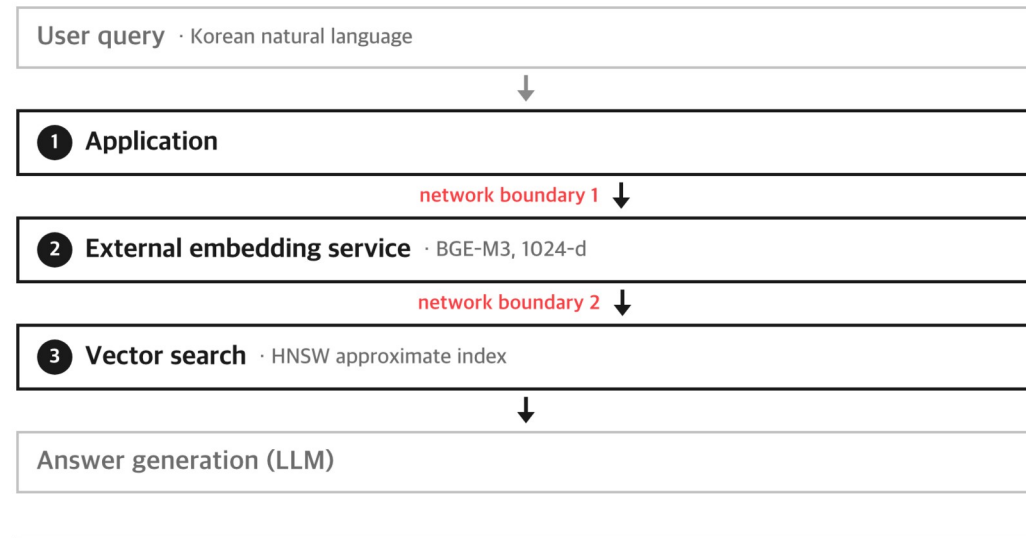


- Load the target store with the vectors the source system has already produced — that is condition B
- A→B differs only in the store, B→C only in the placement — one factor at a time

System Architecture · What Actually Changes

A · Deployed baseline

PostgreSQL + pgvector



Network boundaries

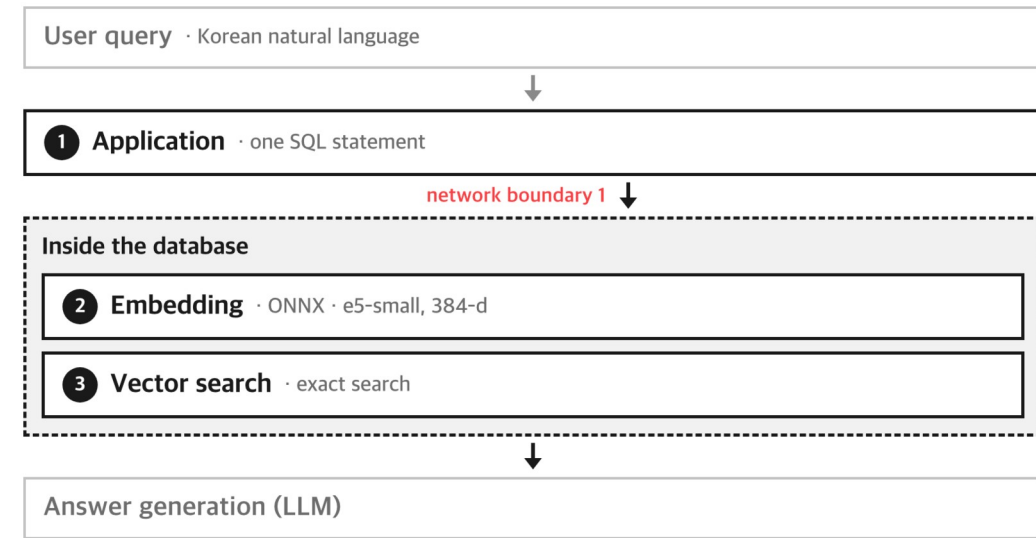
2

External model call

yes

C · Migrated system

Oracle AI Database 26ai



Network boundaries

1

External model call

no

The retrieval rule (vector similarity + metadata filters + source-based prioritization) and its parameters were kept identical across the three conditions.

Experimental Setup

	A · baseline	B · bridge	C · migrated
Store	PostgreSQL + pgvector	Oracle 26ai	Oracle 26ai
Embedding model	BGE-M3 (1024)	BGE-M3 (1024)	e5-small (384)
Placement	external	external	in-database
Vector search	HNSW approx.	exact	exact

Corpus and metrics

- 1,347 review chunks from a Korean product-review service in production · 43 fixed queries
- Retrieval agreement (Jaccard · Spearman · Recall@10) / evidence relevance (0–2 scale) / search latency

Confidence intervals from 10,000 bootstrap resamples. The target database ran on the lowest-tier free instance.

Result ① Store Factor · No Effect Detected

Top-10 Jaccard

0.971

[0.946, 0.992]

Relevance difference

-0.042

$p = 0.523$

Effect size d_z

-0.098

negligible

- Loaded with identical vectors, the two stores returned exactly the same top-10 set for 37 of the 43 queries
- What the baseline missed because of its approximate index amounts to about 1.6%
- A Wilcoxon test on the queries that remain after excluding ties leads to the same conclusion ($p = 0.908$)

Result ② Placement Factor · Significant, Medium-Sized

Top-10 Jaccard

0.175

[0.133, 0.221]

Relevance score

1.270 → 1.014

$p = 0.003$

Effect size d_z

0.473

medium

- Same store and same search method, yet mostly different evidence is retrieved for the same query
- Since no store effect was detected from A to B, virtually all of the quality change is due to the placement factor and the model size limit that comes with it
- Had only an end-to-end comparison been performed, the 20% drop in relevance would have been blamed on the store — and reverting the store would not bring the quality back

But · Claiming “No Difference” Needs Its Own Test

A non-significant p value is not evidence of equivalence

- We ran equivalence tests (TOST) separately over a ± 0.15 point range
- The result was $p = 0.052$ — the lower bound of the confidence interval, -0.172 , falls outside the range, so the significance level was not met

So we report it this way

- Not “equivalence has been established”, but “no effect was detected and its magnitude is bounded to a small range”
- The direction in which the interval extends is the one where the target store scores higher, so there is no basis for concluding that substituting the store degraded quality

Conclusion

Method

- Adding a single bridge condition splits an end-to-end comparison into two single-factor comparisons

What we observed in this service

- No effect of the store factor was detected; the entire observed change originated from the placement factor
- Changing the placement is a trade in governance rather than in performance

A practical decision rule

- If governance of the stored data is the only goal → replacing the store alone is enough
- If the query path must also stay inside the boundary → first confirm that the largest model that can be hosted is sufficient for that domain

The bridge condition makes that confirmation possible before the migration

Limitations and Future Work

Limitations

- A single Korean corpus of 1,347 chunks from one product domain
- Human assessors were not involved in the relevance scoring
- Measurements were taken on an entry-level instance — the latency figures cannot be generalized to higher-tier hardware
- Placement and model size are coupled by construction, and the store result does not establish equivalence

Future work

- A fourth condition that runs the smaller model outside the database would separate placement from model size

Thank you